LiDAR-Guided Hospital Navigation Robot

Final User Documentation

CSE 396 - Computer Engineering Project
Gebze Technical University - Group 10

Muhammet Gökhan ÖZBEK (210104004038)
Samet ALKAN (230104004034)
Hicran KOÇ (220104004074)
Fatih Emre OĞAN (230104004090)
Mervan Deniz YILDIRIM (230104004173)
Ertuğrul ERGÜL (240104004994)
Ömer Faruk GÜRSEL (210104004053)
Sayed Khalilullah HASHİMİ (200104004806)

June 2026

Website: https://lidarnavrobot.github.io/
Demonstration Video: https://www.youtube.com/watch?v=6H9Xc4hRhOg

Abstract

This document is the final user guide for the *LiDAR-Guided Hospital Navigation Robot*, developed by Group 10 for the CSE 396 Computer Engineering Project course. The robot is an autonomous indoor guide platform that uses an RPLidar A2, wheel encoders, an MPU-6050 IMU, and a Raspberry Pi 4 to map indoor corridors, estimate its position, plan a route, and guide a visitor to a selected destination. ROS 2 provides communication between the software modules, while RViz2 is used to display the live LiDAR scan, occupancy map, robot pose, transforms, and planned route. The final prototype uses four driven wheels and four encoder outputs, controlled as a left and right skid-steer pair. This manual explains the final hardware connections, essential software structure, installation, mapping, RViz2 usage, normal operation, and common fault recovery.

Contents

1 Introduction

1.1 Project Scope

Hospitals can be difficult to navigate because visitors must find departments, examination rooms, wards, and service points inside large buildings where GPS is unavailable. This project provides a mobile robot that can operate on a known indoor floor map and guide a visitor to a registered destination.

The robot performs the following main tasks:

1. scans the environment in 360 degrees using the RPLidar A2;
2. creates or loads a 2-D occupancy map of the building;
3. estimates its pose using LiDAR, encoder odometry, and IMU heading data;
4. displays the map, scan, pose, transforms, and path in ROS 2 RViz2;
5. calculates a route to a selected destination using A* path planning;
6. follows the route with differential left/right motor control;
7. detects blocked corridors, stops safely, and resumes or reroutes;
8. optionally receives voice commands and provides spoken feedback.

All critical navigation processing runs locally on the Raspberry Pi 4. The core mapping and navigation functions do not require an active internet connection.

1.2 Final Hardware Configuration Notice

Early planning documents described a two-motor differential-drive platform. The final physical prototype uses a four-wheel chassis with four motor channels and four encoder outputs. The two motors on the left are treated as one logical drive side, and the two motors on the right are treated as the other logical drive side. Therefore, the navigation model remains differential/skid-steer even though four motors are installed.

2 Hardware Architecture

2.1 Processing Nodes

Node	Hardware	Main Role
Main Computer	Raspberry Pi 4	ROS 2, SLAM, RViz2 data, path planning, navigation logic, GPIO control, voice and telemetry
Environment Sensor	RPLidar A2	360-degree distance scanning for mapping, localization, and obstacle detection
Motion Sensors	Four encoder outputs + MPU-6050	Distance, wheel movement, heading correction, and motion feedback
Drive System	Motor driver stage + four DC motors	Left/right skid-steer motion of the mobile platform
User Interface	USB microphone, speaker, RViz2, website/video	Destination input, spoken status, monitoring, and demonstration

Table 1: Main system hardware and roles.

2.2 Power and Ground Rules

- Power the Raspberry Pi from a stable regulated supply suitable for the Pi 4.
- Power the motors through the motor driver from the motor battery; do not power motors from GPIO.
- Connect the Raspberry Pi ground and motor-driver control ground together.
- Keep motor power wiring away from LiDAR, encoder, and I2C signal cables where possible.

- Switch off motor power before changing GPIO connections.

IMPORTANT: Raspberry Pi GPIO pins use 3.3 V logic and are not 5 V tolerant. Encoder or sensor outputs must be reduced to 3.3 V when necessary.

2.3 Final GPIO Pin Assignments

The following table is the authoritative pin map for the final four-motor, four-encoder build. GPIO numbering is given in BCM mode.

Category	Signal	Raspberry Pi GPIO	Physical Pin	Purpose
Encoder	Encoder 1 OUT	GPIO16	Pin 36	Wheel pulse input
Encoder	Encoder 2 OUT	GPIO20	Pin 38	Wheel pulse input
Encoder	Encoder 3 OUT	GPIO21	Pin 40	Wheel pulse input
Encoder	Encoder 4 OUT	GPIO26	Pin 37	Wheel pulse input
Motor 1	M1_IN1	GPIO17	Pin 11	Motor 1 direction input 1
Motor 1	M1_IN2	GPIO27	Pin 13	Motor 1 direction input 2
Motor 2	M2_IN3	GPIO22	Pin 15	Motor 2 direction input 1
Motor 2	M2_IN4	GPIO23	Pin 16	Motor 2 direction input 2
Motor 3	M3_IN1	GPIO24	Pin 18	Motor 3 direction input 1
Motor 3	M3_IN2	GPIO25	Pin 22	Motor 3 direction input 2
Motor 4	M4_IN1	GPIO5	Pin 29	Motor 4 direction input 1
Motor 4	M4_IN2	GPIO6	Pin 31	Motor 4 direction input 2

Table 2: Final Raspberry Pi motor and encoder pin mapping.

2.4 Other Hardware Interfaces

Device	Connection	Operational Note
RPLidar A2	USB serial connection to Raspberry Pi	Confirm the serial device before starting ROS 2; keep the LiDAR level and unobstructed.
MPU-6050	I2C: SDA GPIO2 / Pin 3, SCL GPIO3 / Pin 5	Default I2C address is normally 0x68; enable I2C in Raspberry Pi configuration.
USB Microphone	Raspberry Pi USB	Used by the speech-to-text module for destination commands.
Speaker / Amplifier	Raspberry Pi audio output or USB audio	Used for departure, obstacle, and arrival messages.
Motor Driver	GPIO direction inputs and external motor supply	Motor current must not pass through the Raspberry Pi.

Table 3: Additional hardware connections.

NOTE: Single-output encoders provide pulse counts but not independent direction information. Wheel direction is inferred from the active motor command.

3 Software Architecture

3.1 ROS 2 Data Flow

The software is divided into cooperating functions rather than one large program. Sensor drivers publish measurements, the localization system estimates the pose, the planner creates a route, and the motion controller converts the route into left/right motor commands.

LiDAR + Encoders + IMU → Mapping / Localization → A* Planner → Path Controller → Motor Driver

Typical ROS 2 information used by the system includes:

Information	Typical ROS 2 Topic / Frame	Purpose
Laser scan	`/scan`	Raw 360-degree LiDAR distance measurements
Occupancy map	`/map`	Free, occupied, and unknown cells of the floor map
Odometry	`/odom`	Short-term robot movement estimated from wheel feedback
Transforms	`map -> odom -> base_link -> laser`	Connects map, robot body, and LiDAR coordinate frames
Planned path	`/plan` or planner path topic	Route produced between the current pose and destination
Navigation goal	RViz2 "2D Goal Pose"	Destination selected by the operator during testing

Table 4: Main ROS 2 data displayed or consumed during operation.

3.2 Mapping, Localization, and RViz2

During mapping, the robot moves through the environment while the LiDAR scan is converted into a 2-D occupancy grid. Encoder odometry and the MPU-6050 help estimate motion between scans, while LiDAR scan matching corrects drift. The completed map is saved as a `.pgm` image and a `.yaml` metadata file.

RViz2 is the main engineering visualization tool. Set the **Fixed Frame** to `map` and add:

- **Map** for the occupancy grid;
- **LaserScan** for live RPLidar points;
- **TF** for frame relationships;
- **Odometry** or **Pose** for the estimated robot position;
- **Path** for the calculated navigation route;
- **RobotModel** when the final URDF is available.

A healthy RViz2 view shows the laser points aligned with mapped walls and the robot moving on the map without sudden jumps. Misaligned scans usually indicate incorrect TF frames, LiDAR mounting, or odometry direction.

3.3 Navigation and Obstacle Handling

A* calculates a collision-free route on the saved occupancy map. The controller then steers toward each waypoint by changing the relative speed of the left and right drive sides. During movement, the forward LiDAR region is checked continuously. An obstacle closer than approximately 0.50 m causes the robot to stop. If the route clears, movement resumes; if the blockage persists, the navigation logic selects a safer route or recovery motion before continuing.

3.4 Voice and Digital Visualization

The voice module can convert a spoken destination into a navigation goal and announce events such as departure, obstacle waiting, arrival, and failure. The digital visualization module can mirror the robot pose and status for operator monitoring. These functions support the user experience but are not required for basic LiDAR mapping and RViz2 navigation tests.

4 Installation and Setup

4.1 Raspberry Pi and ROS 2 Environment

Use the Raspberry Pi OS / Ubuntu image and ROS 2 distribution selected by the project team. The following commands show the standard workspace preparation sequence used before starting the robot software:

```
# Load the ROS 2 environment
source /opt/ros/humble/setup.bash

# Build the submitted workspace
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash

# Confirm that ROS 2 can see the system
ros2 node list
ros2 topic list
```

The final launch command depends on the package and launch-file names in the submitted workspace:

```
# Mapping mode
ros2 launch <robot_package> <mapping_launch_file>.py

# Saved-map navigation mode
ros2 launch <robot_package> <navigation_launch_file>.py
```

Replace the values in angle brackets with the names used in the final project folder. This manual does not invent a repository or package name that is not part of the submitted system.

4.2 Basic Device Checks

Before operating the complete system, verify each important interface separately:

```
# LiDAR / USB serial devices
ls /dev/ttyUSB* /dev/ttyACM*

# I2C devices; MPU-6050 normally appears at address 68
sudo i2cdetect -y 1

# ROS 2 LiDAR data
ros2 topic echo /scan --once

# Start RViz2
rviz2
```

4.3 Hardware Wiring Checklist

1. Confirm all motor and encoder GPIO connections using Table 2.
2. Confirm that encoder outputs do not exceed 3.3 V.
3. Connect the MPU-6050 to the Raspberry Pi I2C pins and enable I2C.
4. Connect the RPLidar A2 by USB and confirm that its rotating head can move freely.
5. Join the Raspberry Pi control ground and motor-driver ground.
6. Keep the robot lifted from the floor for the first motor-direction test.
7. Confirm that left and right commands rotate the correct physical wheel groups.
8. Place the robot on a flat floor only after the stop command has been verified.

4.4 RViz2 Initial Configuration

1. Start RViz2 after the mapping or localization nodes are running.

2. Set **Fixed Frame** to `map`.
3. Add **Map** and select `/map`.
4. Add **LaserScan** and select `/scan`.
5. Add **TF**, **Odometry/Pose**, and **Path**.
6. Save the RViz2 configuration for later demonstrations.

5 Operational Procedures

5.1 Startup Checklist

1. Charge the Raspberry Pi supply and motor battery.
2. Inspect motor, encoder, LiDAR, and I2C cables.
3. Place the robot in an open starting area on the mapped floor.
4. Power the Raspberry Pi first, then enable motor power.
5. Source the ROS 2 workspace and start the required launch file.
6. Open RViz2 and confirm that `/scan`, `/map`, and TF data are visible.
7. Do not start autonomous navigation until the robot pose is correct on the map.

5.2 Creating and Saving a Map

1. Start the LiDAR driver and mapping launch file.
2. Open RViz2 and verify that live scan points match nearby walls.
3. Move the robot slowly through corridors and junctions; avoid rapid turns and wheel slip.
4. Return near the starting position when possible to improve map consistency.
5. Save the completed occupancy map:

```
mkdir -p ~/maps
ros2 run nav2_map_server map_saver_cli -f ~/maps/hospital_floor
```

The command produces `hospital_floor.pgm` and `hospital_floor.yaml`. Keep both files together because the YAML file stores map scale, origin, and image information.

5.3 Navigation on a Saved Map

1. Start the saved-map localization and navigation launch file.
2. Confirm that the correct map appears in RViz2.
3. Set or verify the robot's initial pose if required by the localization system.
4. Select **2D Goal Pose** in RViz2 and click the desired destination and heading.
5. Watch the planned path before allowing movement.
6. Keep an operator near the emergency stop during the test.
7. At the destination, confirm that the robot stops and the status changes to goal reached.

5.4 Safe Shutdown

Stop autonomous movement before closing ROS 2 nodes. Disable motor power, stop the launch process with `Ctrl+C`, shut down the Raspberry Pi normally, and disconnect the batteries only after the operating system has stopped.

6 Troubleshooting

- **LiDAR does not appear:** check the USB cable and serial device, then confirm user permission for the serial port.
- **RViz2 shows no scan:** verify the LiDAR node and `/scan` topic; select the correct topic in LaserScan display.
- **Map is empty or distorted:** move slowly, verify TF frames, and check that the LiDAR is rigidly mounted and level.
- **Robot moves in the wrong direction:** stop immediately and reverse the affected motor input pair or correct the software direction setting.
- **Encoder count stays zero:** check the correct BCM pin, ground, voltage level, and GPIO edge-detection configuration.
- **Pose drifts or jumps:** inspect wheel slip, encoder scaling, IMU orientation, and LiDAR scan alignment.
- **Motors reset the Raspberry Pi:** use separate regulated power paths, common ground, shorter wiring, and motor-noise suppression.
- **Robot refuses a goal:** confirm that the goal is in free map space and that the planner has sufficient clearance from walls.

7 Design Notes and Known Constraints

1. **Four-Motor Skid Steering:** The final robot has four motors, but navigation commands are generated for logical left and right sides. Unequal floor grip can cause turning and odometry errors.
2. **Encoder Direction:** Each encoder provides one output signal. Direction is inferred from the commanded motor direction; an encoder alone cannot identify reverse motion if the wheel is pushed externally.
3. **LiDAR Line of Sight:** The 2-D LiDAR measures one horizontal plane. Objects entirely above or below that plane may not be detected. Glass, mirrors, dark surfaces, and direct sunlight can also reduce measurement quality.
4. **Map Dependence:** Reliable navigation requires a map that matches the physical environment. Large furniture changes, closed corridors, or a moved starting position may require remapping or re-localization.
5. **Localization Drift:** Encoder and IMU estimates accumulate error during wheel slip or rapid turns. LiDAR scan matching and known corridor features are used to correct this drift.
6. **GPIO Safety:** Raspberry Pi GPIO is 3.3 V only. Incorrect voltage, short circuits, or motor current entering a GPIO can permanently damage the computer.
7. **Power Separation:** Motor current changes can generate voltage drops and electrical noise. The motor supply and Raspberry Pi supply should be stable and properly grounded.
8. **Emergency Supervision:** The prototype must be supervised during tests. An operator must be able to cut motor power immediately if localization, planning, or motor direction is incorrect.
9. **ROS 2 Names:** Topic and launch-file names may be remapped in the final workspace. RViz2 displays must be connected to the topic names actually published by the running nodes.
10. **Voice Interface:** Speech recognition can be affected by corridor noise, pronunciation, and microphone placement. The robot should confirm the selected destination before beginning autonomous movement.